

Quantum Computing Performance Analysis: CUDA-Q Backends

Not So Slow Slugs

March 9, 2026

Task 1: Precision Performance Analysis

We found performance on double precision (FP64) to be slightly slower than single precision (FP32). As the data size doubles, the computational overhead on the GPU increases, resulting in longer total calculation times.

Table 1: Sampling Performance: Single vs. Double Precision

Configuration	Single Precision (s)	Double Precision (s)
1 GPU, 31 Qubits	0.4522	0.6989
1 GPU, 32 Qubits	0.5651	0.6082
2 GPU, 32 Qubits	0.8212	0.8498
2 GPU, 33 Qubits	1.5176	1.8135
Tensor (1 GPU, 90 Qubits)	26.6915	29.8744

Task 3: Scaling Limits for Tensor Backends

We analyzed the scalability of `tensornet` and `tensornet-mps` on NVIDIA V100 and H100 GPUs. While both backends handle larger qubit counts than State Vector simulators, they worked a little differently.

V100 Performance Data

The time to compute jumped a fair bit from 300 qubits to 400 qubits on the `tensornet` and `tensornet-mps`. Interestingly, even though the mps system is supposed to be faster because it's making approximations about the system (especially with such a low entanglement system like this one) it still underperformed compared to the normal `tensornet` system.

tensornet-mps			tensornet		
Qubits	Time (s)	Qubits/s	Qubits	Time (s)	Qubits/s
100	58.64	1.70	100	47.43	2.10
200	227.65	0.87	200	176.76	1.13
300	498.19	0.60	300	355.04	0.84
400	935.83	0.42	400	756.05	0.52

H100-80 Tensor Performance

As expected, the H100 GPU has much higher performance than the V-100. For the H100-80 using only tensor, we observed the best performance at 298 qubits, with performance starting to slow down after that. We observed slightly slower performance with the tensor-mps and it peaked in effectiveness at about 240 qubits. We found that there was nearly a quadratic relationship between qubit count and execution time.

Table 2: H100-80 `tensor`net — Coarse Sweep

Qubits	Time (s)	Qubits²/s
250	206.02	303
275	229.65	329
280	249.75	314
285	246.36	330
290	260.48	323
295	261.22	333
300	265.98	338
305	293.42	317
325	345.37	306

Table 3: H100-80 `tensor`net — Fine-Grained Sweep (295–304 Qubits)

Qubits	Time (s)	Qubits²/s
295	255.71	340
296	257.95	340
297	259.73	340
298	259.29	342
299	262.61	340
300	264.55	340
301	280.89	323
302	276.38	330
303	291.29	315
304	284.85	324

H100-80 Tensor-MPS Performance

Table 4: H100-80 `tensor`net-mps — Coarse Sweep

Qubits	Time (s)	Qubits²/s
250	225.96	277
350	456.54	268
450	786.01	258
550	1235.82	245

Table 5: H100-80 `tensor`net-mps — Fine-Grained Sweep (200–300 Qubits)

Qubits	Time (s)	Qubits²/s
200	146.15	274
210	165.82	266
220	178.39	271
230	190.20	278
240	196.20	294
250	217.69	287
260	246.27	274
270	262.02	278
280	283.17	277
290	305.05	276
300	314.05	287

Task 4: Backend Selection Strategy

Based on empirical data, backend choice should follow these criteria:

- **State Vector** (`nvidia`): Best for low qubit counts where exact results and high entanglement are required.
- **Exact Tensor** (`tensornet`): Optimal for medium to high qubit counts with low-to-moderate entanglement.
- **MPS Tensor** (`tensornet-mps`): Necessary for massive systems where memory efficiency is prioritized over absolute precision, utilizing approximations to maintain performance. (Even if we weren't directly seeing this in the data we were collecting.)

Task 5: Multi-GPU and Asynchronous Execution

To optimize large-scale expectation value calculations, we utilized `observe_async` and Hamiltonian chunking to distribute workloads across available QPUs.

```
1 # Hamiltonian splitting logic
2 terms = [term for term in H]
3 chunk_size = 10
4 chunks = [terms[i:i+chunk_size] for i in range(0, len(terms), chunk_size)]
5
6 for i, chunk in enumerate(chunks):
7     H_chunk = sum(chunk)
8     qpu_id = i % ngpu # Alternate GPU assignment
9     future = cudaq.observe_async(ansatz, H_chunk, n_qubits, qpu_id=qpu_id)
10    results.append(future)
```

Listing 1: Asynchronous Hamiltonian Decomposition

```
1 # Multi-QPU Batch Processing
2 h_ops = []
3 for i, chunk in enumerate(chunks):
4     if i % len(allTargets) != index:
5         continue
6     h_op = z_power_n(chunk) + x_power_n(chunk) + y_power_n(chunk)
7     h_ops.append(h_op)
8
9 for i, hchunk in h_ops:
10    result = cudaq.observe_async(ghz_kernel, hchunk, qpu_id=i%len(allTargets))
11    results.append(result)
```

Listing 2: Batch Expectation Calculation

Task 6: GHZ Linear Topology Performance

We implemented a linear GHZ circuit by modifying the given `ghz.py` file, specifically in the `ghz_kernel` section. Then, we compared execution times across backends. A linear chain of CNOT gates was used to propagate entanglement.

```
1 # Chaining qubits in a linear topology
2 for i in range(qubit_count - 1):
3     x.ctrl(qubits[i], qubits[i + 1]) # Chaining current to next with a CNOT
```

Listing 3: Linear Entanglement Logic

Table 6: Linear GHZ Performance Benchmarks

Backend	Code	Qubits	Time (s)
qpp-cpu (1 RM node)	ghz_linear.py	31	617.31
nvidia (1 GPU)	ghz_linear.py	31	0.81
mgpu (2 GPU)	ghz_linear.py	32	2.18
tensornet (1 GPU)	ghz_linear.py	90	28.50